🧭 Project Experience: Mission Control – Multi-Soldier Task Orchestrator

---

Approached towards Problem and Solution

I began by visualizing the problem as a **mission management system**:

- A **Commander service** assigns tasks and manages mission states.
- Multiple **Soldier workers** receive and execute those missions asynchronously.
- A **message queue (RabbitMQ)** ensures decoupled communication between them.
- **Redis** stores mission progress and queue information for each soldier.

I focused on building the project incrementally — starting with simple APIs, then introducing JWT-based security, and finally containerizing everything.
Automated scripts (.ps1 and .sh) were added later to simulate multiple mission assignments and monitor real-time progress.

Challenges

This project came with multiple challenges that improved my debugging and architectural thinking:

- **RabbitMQ Connection Failures:**
  Solved by using Docker service hostnames (e.g., rabbitmq, commander) instead of localhost.
- **Redis WRONGTYPE Errors:**
  Occurred when different data types shared the same key. Fixed by defining clear key patterns like mission:{id} and soldier_queue:{id}.
- **Queued Tasks Not Executing:**
  Fixed by implementing a **per-soldier queue mechanism** in Redis and enhancing the worker logic to pick the next mission automatically after completing the previous one.
- **Token Refresh Overwrites:**
  Adjusted token caching so that tokens only refresh after expiry instead of generating new ones on every request.

These issues taught me how o trace and fix problems across multiple containers effectively.

Mission Creation

- The system first checks Redis to verify whether the soldier is currently active or already assigned to a mission.
- If the soldier is free, a new JWT token is issued and attached to the mission for secure validation.
- The mission is stored in Redis with a status of QUEUED and published to RabbitMQ (orders_queue).
- The assigned Soldier (running as a separate worker container) receives the task from the queue and begins execution automatically.

Mission Tracking

- It continuously reports status updates (IN_PROGRESS, COMPLETED, or FAILED) to RabbitMQ (status_queue).
- The Commander listens to these updates using a background listener and reflects changes in Redis instantly.
- Redis serves as the single source of truth for all mission states, allowing anyone to fetch a mission's status via /missions/{mission_id}.
- Additionally, soldiers queued for future missions automatically start their next task once the previous one completes — ensuring seamless and continuous execution.

This mechanism creates a **real-time, event-driven feedback loop** where every mission can be tracked end-to-end, from assignment to completion, through the Commander's API dashboard or test scripts.

Key Takeaways

This project strengthened my ability to **design, debug, and optimize distributed systems**.
I learned how crucial structured logging, retry mechanisms, and proper container networking are for stable microservice communication.